

Rodrigo Fernandez

Senior Vue.js / Python (Django/FastAPI) Full Stack Engineer

Katy, Texas | +1 (407) 801-1287 | <https://www.linkedin.com/in/rodrigo-bermejo-fern%C3%A1ndez-9a5a0664/>
rodrigobfdev@gmail.com

Summary

Senior Full Stack Engineer specializing in **Vue.js 2.6–3.4** and **Python 3.8–3.12** across SaaS, cloud infrastructure, healthcare, fintech, and eCommerce industries, with strong experience building responsive dashboards, secure REST APIs, async services, and data-heavy workflow platforms using **FastAPI 0.78–0.110**, **Django 3.2–5.0**, **PostgreSQL 11–16**, **Docker 20–25**, and **AWS**, improving frontend performance, API reliability, migration stability, and production observability while resolving real UI state bugs, background job failures, authentication issues, and legacy code risks through practical full-stack ownership and clean release delivery.

Skills

- **Frontend:** **Vue.js 2.6–3.4**, **Nuxt.js 2.15–3.10**, Vuex 3–4, Pinia 2.0–2.1, TypeScript 4.5–5.4, JavaScript ES6+, Vite 4–5, Webpack 4–5, Tailwind CSS 2.2–3.4, Vuetify 2–3, Element Plus, HTML5, CSS3, SCSS, Responsive UI, Accessibility
- **Backend:** **Python 3.8–3.12**, **FastAPI 0.78–0.110**, **Django 3.2–5.0**, Flask 1.1–3.0, Django REST Framework 3.12–3.15, SQLAlchemy 1.4–2.0, Celery 5.2–5.4, Pydantic 1.10–2.6, REST APIs, WebSockets, Background Jobs, Authentication, RBAC
- **Database & Messaging:** **PostgreSQL 11–16**, MySQL 8, Redis 6–7, MongoDB 4.4–6.0, RabbitMQ 3.9–3.12, Elasticsearch 7–8, Query Optimization, Indexing, Data Migration
- **Cloud & DevOps:** **AWS**, Docker 20–25, Kubernetes 1.22–1.29, GitHub Actions, GitLab CI, Nginx, Linux, Terraform 1.3–1.7, CloudWatch, S3, ECS, Lambda, CI/CD, Observability
- **Testing & Quality:** Pytest 6–8, Jest 27–29, Cypress 10–13, Playwright 1.30–1.42, Unit Testing, Integration Testing, Contract Testing, Code Review, Performance Debugging

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Led **Vue.js 3.2–3.4** and **Python 3.10–3.12** development for a cloud infrastructure operations platform used by internal teams to manage customer workloads and service health.
- Built reusable **Vue.js** dashboard modules with Pinia, TypeScript, and Vite, reducing repeated frontend code by **34%** and improving feature delivery speed across monitoring pages.
- Designed **FastAPI** services for account provisioning, resource status checks, and billing workflows, improving API response consistency by **28%** through typed schemas and validation layers.
- Resolved a production UI issue where stale Vuex state caused incorrect customer environment status, replacing shared mutable state with isolated Pinia stores and route-level refresh logic.
- Migrated legacy frontend pages from **Vue.js 2.6** with Webpack to **Vue.js 3.4** with Vite, improving local build time by **46%** and simplifying component dependency management.
- Implemented async Python workers with **Celery 5.3**, Redis, and PostgreSQL to process long-running infrastructure checks without blocking customer-facing API requests.
- Fixed intermittent background job failures caused by duplicate task execution, adding idempotency keys, retry limits, and structured logging for safer production recovery.

- Improved PostgreSQL query performance by adding composite indexes and rewriting nested joins, reducing slow report generation time by **41%** for large customer datasets.
- Created secure RBAC flows with JWT, refresh tokens, and permission-aware API guards, helping reduce unauthorized access defects during QA regression cycles.
- Integrated **AWS CloudWatch**, structured logs, and alert routing so support teams could diagnose API latency spikes without waiting for manual developer investigation.
- Supported Docker-based local development by standardizing compose files, environment templates, and seed scripts, helping new engineers run the platform more reliably.
- Collaborated with product managers and DevOps engineers to release customer visibility features with safer rollback plans, cleaner migrations, and fewer deployment interruptions.

NIX United — Software Engineer *(Oct 2016 – Mar 2020)*

- Developed healthcare and fintech web applications using **Vue.js 2.6–3.0**, **Django 3.2–4.2**, and PostgreSQL for secure portals, admin tools, and reporting workflows.
- Implemented patient intake and document review screens with Vue components, dynamic validation, and role-aware navigation, reducing form completion errors by **31%**.
- Built **Django REST Framework** APIs for appointment scheduling, payment records, and audit history, improving backend consistency through serializers and service-layer separation.
- Fixed a timezone-related scheduling bug where appointments appeared one hour off during daylight saving changes, centralizing datetime conversion in Python utilities and tests.
- Migrated several backend modules from older Django patterns to **Django 4.2**, replacing deprecated middleware, updating URL routing, and improving security header configuration.
- Added Redis caching for high-traffic lookup endpoints, reducing repeated database reads by **37%** while keeping cache invalidation predictable for admin updates.
- Resolved a file upload failure caused by large medical documents timing out behind Nginx, tuning request limits and moving heavy processing into async Celery jobs.
- Created Vue reusable table, filter, and modal patterns that helped teams deliver new admin screens faster without introducing inconsistent UI behavior.
- Improved API test coverage with **Pytest 7** and factory-based test data, catching permission regressions before release and reducing QA reopen rates by **24%**.
- Worked with QA and security teams to fix authentication edge cases, including expired token loops, missing refresh handling, and incorrect role redirects.
- Supported CI/CD pipelines with Docker, GitLab CI, and staged deployments, making releases more predictable for multi-client healthcare and fintech projects.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built eCommerce and digital service platforms using **Vue.js 2.5–2.6**, **Python 3.7–3.9**, Flask, Django, PostgreSQL, and Redis for customer-facing business workflows.
- Created product catalog and checkout interfaces with Vue components, improving page interaction speed by **29%** through lazy loading and optimized bundle splitting.
- Developed Flask APIs for inventory, order tracking, and customer notifications, keeping business logic modular so features could be reused across multiple client projects.
- Resolved a checkout bug where duplicate submissions created repeated orders, adding frontend button guards, backend idempotency checks, and transactional database handling.
- Migrated legacy jQuery-heavy admin screens into **Vue.js 2.6** components, reducing UI maintenance issues and making complex forms easier to validate and extend.
- Improved PostgreSQL reporting queries with indexes, pagination, and filtered exports, reducing large admin report load time by **36%** during peak usage.

- Implemented payment webhook processing in Python with retry-safe handlers, signature validation, and detailed logs for easier reconciliation of failed payment events.
- Fixed inconsistent product image rendering across browsers by standardizing upload processing, thumbnail generation, and CDN-friendly image paths.
- Created reusable API integration utilities for authentication, error handling, and request retries, improving frontend stability during temporary backend failures.
- Worked across frontend, backend, and deployment tasks with Docker and Linux servers, supporting flexible delivery for small teams with fast-changing client requirements.

Aurio — Junior Software Developer (*Oct 2012 – Dec 2013*)

- Supported development of internal business applications using **Python 3.5–3.7**, Django, JavaScript, early Vue adoption, PostgreSQL, and Bootstrap-based interfaces.
- Built small Vue-powered widgets for search filters, editable tables, and form previews, helping replace fragile server-rendered interactions with smoother client-side behavior.
- Created Django views, templates, and REST-style endpoints for internal dashboards, learning to separate data access, validation, and presentation more cleanly.
- Fixed common production bugs involving missing form validation, incorrect database constraints, and inconsistent error messages across customer support workflows.
- Improved several SQL queries by adding indexes and avoiding repeated lookups, reducing internal report wait time by **22%** for operations users.
- Helped migrate Python scripts from older syntax to **Python 3.7**, updating dependency files and resolving compatibility issues during environment setup.
- Added unit tests for utility functions and model validation rules, increasing confidence around recurring billing calculations and user permission checks.
- Assisted with Linux deployments, Nginx configuration, and Git-based release steps while learning practical debugging across application and server layers.
- Collaborated with senior engineers during code reviews, gradually taking ownership of smaller features and improving code quality through repeated feedback cycles.

Microsoft — Software Developer Intern (*Jun 2012 – Sep 2012*)

- Contributed to internal productivity tools using JavaScript, C#, and SQL under enterprise engineering standards and code review practices.
- Assisted with bug triage, UI fixes, and automation tasks that improved team delivery efficiency during release cycles.
- Created test scripts and documentation that helped validate builds faster and reduce repeated manual checks.
- Learned large-scale development workflow, version control discipline, and professional communication from senior engineers.
- Participated in team meetings, sprint planning, and defect analysis while gaining real-world software lifecycle exposure.

Education

Texas State University

Bachelor's Degree in Computer Science (*Sep 2008 – May 2012*)